

Joins and the fan-out trap

When you combine two tables, how do you know the total is still right?

BY THE END OF TODAY, YOU CAN

Class objectives

- 01 Explain why tables are split
 - 02 Run the Five Checks before a join
 - 03 Say which rows INNER and LEFT JOIN keep
 - 04 Audit a join and fix an inflated total
 - 05 Find unmatched rows with LEFT JOIN and IS NULL
-

Warm-up

Five questions on D04, ungraded.

Suggested time • 10 minutes

PART 02

Why the data is split, and how it joins back

Yesterday's open question needs two tables at once.

Why the data is split

One customer, many orders, and why the name is stored only once.

Suggested time • 8 minutes

[labs/D05/01-Ins-Why-Tables-Split/README.md](https://github.com/DataCamp/labs/blob/main/D05/01-Ins-Why-Tables-Split/README.md)

WHAT IF

If every order carried the customer's city

Lina moves from Amman to Aqaba. How many rows do you change?

order_id	status	customer_name	city
1001	completed	Omar Nasser	Irbid
1002	cancelled	Lina Haddad	Amman
1003	completed	Omar Nasser	Irbid
1004	completed	Omar Nasser	Irbid
1005	completed	Lina Haddad	Amman

One customer, many orders

C-118 has two orders. Each order has exactly one customer.

Primary key

Foreign key

customer_id	customer_name	1	many	order_id	customer_id	status
C-118	Lina Haddad			1001	C-204	completed
C-204	Omar Nasser			1002	C-118	cancelled
C-377	Rana Khoury			1003	C-204	completed
C-401	Omar Nasser			1004	C-401	completed
C-455	Sami Odeh			1005	C-118	completed

DEFINITION

A join puts the columns of two tables side by side for one query, matching rows on a key; neither table changes.

The Five Checks

Run them out loud before every join this week, and in Assignment 1.

Suggested time • 7 minutes

[labs/D05/01-Ins-Why-Tables-Split/README.md](https://github.com/DataCamp/labs/blob/main/D05/01-Ins-Why-Tables-Split/README.md)

The Five Checks before a join

-
- 01 What is one row in each table?
 - 02 Which key joins them?
 - 03 Is the key unique on the “one” side?
 - 04 How many rows should come out?
 - 05 What happens to rows with no match?
-

Is the key unique on the one side?

Equal counts mean every customer_id appears once in customers.

```
SELECT COUNT(*) AS customer_rows,  
       COUNT(DISTINCT customer_id) AS different_ids  
FROM d01.customers;
```

→ 5 · 5: unique

INNER JOIN and LEFT JOIN on d01

Predict 5 rows first. Checkpoint: C-377 and C-455 appear only in the LEFT JOIN.

Suggested time • 15 minutes

[labs/D05/02-Evr-Inner-and-Left/README.md](https://github.com/DataCamp/labs/blob/main/D05/02-Evr-Inner-and-Left/README.md)

AS o gives the table a short name

Both tables have customer_id, so **o.** and **c.** say which one.

```
SELECT o.order_id, o.status, c.customer_name, c.city
FROM d01.orders AS o
INNER JOIN d01.customers AS c
    ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

INNER JOIN keeps the rows that match

ON names the key on each side. Every order finds one customer.

```
SELECT o.order_id, o.status, c.customer_name, c.city
FROM d01.orders AS o
INNER JOIN d01.customers AS c
ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

→ 5 rows

customers LEFT JOIN orders

LEFT JOIN keeps every customer

7 rows. C-377 and C-455 have no orders, so their order columns are NULL.

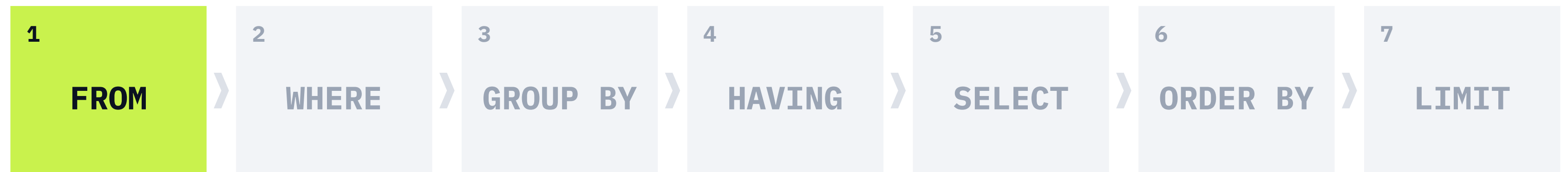
customer_id	customer_name	order_id	status
C-118	Lina Haddad	1002	cancelled
C-118	Lina Haddad	1005	completed
C-204	Omar Nasser	1001	completed
C-204	Omar Nasser	1003	completed
C-377	Rana Khoury	NULL	NULL
C-401	Omar Nasser	1004	completed
C-455	Sami Odeh	NULL	NULL

The order a query runs in

YOU WRITE

SELECT FROM WHERE GROUP BY HAVING ORDER BY LIMIT

BIGQUERY RUNS



JOIN is part of **FROM**: the joined rows exist before WHERE runs.

Audit every join

Rows, distinct keys and the total, before and after.

How many completed orders?

The same question, before and after a join.

Suggested time • 8 minutes

[labs/D05/03-Evr-Count-Completed-Orders/README.md](#)

How many completed orders?

On orders

4

One row is one order

After joining order_items

10

One row is one product on one order

COUNT(DISTINCT o.order_id) after the join: 4 again.

Join Nakheel orders to customers

3,000 orders. Count them again after INNER JOIN and after LEFT JOIN.

Suggested time • 10 minutes

[labs/D05/04-Evr-Orders-to-Customers/README.md](https://github.com/DataCamp/labs/blob/main/D05/04-Evr-Orders-to-Customers/README.md)

INNER JOIN drops one order

orders

3,000

INNER JOIN customers

2,999

LEFT JOIN customers

3,000

Primary key

Foreign key

order_id	customer_id	order_date	status	order_total
101664	C-0999	2025-11-14	completed	938.00

C-0999 is not in customers.

STUDENT DO

Activity: Audit the join

In this activity, you will join Nakheel orders to their order lines and find which number the join changed.

Suggested time

15 minutes

[labs/D05/05-Stu-Join-Audit/README.md](#)

Activity: Audit the join

INSTRUCTIONS

1. On orders: rows, different orders, SUM(order_total).
 2. The same three after orders INNER JOIN order_items.
 3. After the join, sum qty * price instead.
- First, the Five Checks: which is the “one” side?

HINT

order_total is stored once per order. How often does it appear after the join?

BONUS

Which order had its total copied onto the most rows?

Time's up. Let's review.

Audit the join

Solved: `labs/D05/05-Stu-Join-Audit/Solved/join_audit.sql`

Which number did the join change?

SUM(i.qty * i.price) after the join: 340,370.00. The money is all there, summed at the wrong grain.

	Rows	Different orders	SUM(order_total)
orders	3,000	3,000	340,370.00
orders JOIN order_items	7,516	3,000	1,067,249.00

DEFINITION

Fan-out is what happens when a join to the “many” side copies each “one”-side row, and every amount stored on it.

Fix it by choosing the grain

Sum each amount at the table where it lives.

Sum each amount where it lives

Two fixes for the inflated total, and one rule.

Suggested time • 10 minutes

[1abs/D05/06-Ins-Sum-Where-It-Lives/README.md](https://github.com/101labs/D05/06-Ins-Sum-Where-It-Lives/README.md)

INFLATED

order_total, summed after the join

order_total lives on orders. The join copied it onto every line.

```
SELECT o.channel,  
       ROUND(SUM(o.order_total), 2) AS revenue_inflated  
FROM nakheel.orders AS o  
INNER JOIN nakheel.order_items AS i  
  ON o.order_id = i.order_id  
WHERE o.status = 'completed'  
GROUP BY o.channel  
ORDER BY o.channel;
```

→ **online 351,733.50 · store 637,238.00**

FIX 1

Sum the amount stored on each line

qty * price lives on order_items, the grain of the joined rows.

```
SELECT o.channel,  
       ROUND(SUM(i.qty * i.price), 2) AS revenue  
FROM nakheel.orders AS o  
INNER JOIN nakheel.order_items AS i  
  ON o.order_id = i.order_id  
WHERE o.status = 'completed'  
GROUP BY o.channel  
ORDER BY o.channel;
```

→ **online 110,685.50 · store 205,338.00**

Sum an amount at the table where it lives, or
prove the join did not change that table's grain.

COUNT(*) against COUNT(column) after a LEFT JOIN

Why a customer with no orders counts as 1.

Suggested time • 10 minutes

[labs/D05/07-Evr-Count-After-Left-Join/README.md](#)

COUNT(*) counts the empty row too

COUNT(o.order_id) skips the NULL that the LEFT JOIN added.

```
SELECT c.customer_id,  
       COUNT(*) AS rows_counted,  
       COUNT(o.order_id) AS orders  
FROM nakheel.customers AS c  
LEFT JOIN nakheel.orders AS o  
  ON c.customer_id = o.customer_id  
GROUP BY c.customer_id  
ORDER BY orders, c.customer_id;
```

→ **C-1013: 1 row counted, 0 orders**

Activity: Never ordered, never sold

In this activity, you will find the customers who never ordered and the products never sold, with LEFT JOIN and IS NULL.

Suggested time

8 minutes

[labs/D05/08-Stu-Never-Ordered/README.md](#)

Activity: Never ordered, never sold

INSTRUCTIONS

1. Which customers signed up but never placed an order?
 2. Which products have never sold?
- Before each: which table goes on the left?

HINT

After a LEFT JOIN, test the right-hand table's key: `o.order_id IS NULL`.

BONUS

How many never-ordered customers are in each city, as typed?

Time's up. Let's review.

Never ordered, never sold

Solved: `labs/D05/08-Stu-Never-Ordered/Solved/never_ordered.sql`

RIGHT and FULL joins, and a preview of D06

Recognition only. Cut first if we are late.

Suggested time • 8 minutes

[labs/D05/09-Ins-Right-Full-Subquery/README.md](#)

IF TIME ALLOWS

RIGHT and FULL, for recognition

RIGHT is LEFT written backwards. FULL keeps the 39 customers without orders and the orphan order.

Join	Rows
customers LEFT JOIN orders	3,038
orders RIGHT JOIN customers	3,038
customers FULL OUTER JOIN orders	3,039

D01's answer, in SQL

The category revenue you worked out by hand on the first day.

D01's answer, in SQL

Accessories, 166.00, worked out by hand on D01. Now as a query.

Suggested time • 20 minutes

[labs/D05/10-Evr-D01-Category-Revenue/README.md](#)

Two joins, nothing changed

12 rows and 487.00 before and after both joins: each line finds one order and one product.

```
SELECT p.category,  
       SUM(i.qty * i.price) AS revenue  
FROM d01.order_items AS i  
INNER JOIN d01.orders AS o ON i.order_id = o.order_id  
INNER JOIN d01.products AS p ON i.product_id = p.product_id  
WHERE o.status = 'completed'  
GROUP BY p.category  
ORDER BY revenue DESC;
```

Completed orders, by category

The same as D01. Without WHERE, Shoes wins with 195.00: the cancelled order again.

```
SELECT p.category,  
       SUM(i.qty * i.price) AS revenue  
FROM d01.order_items AS i  
INNER JOIN d01.orders AS o ON i.order_id = o.order_id  
INNER JOIN d01.products AS p ON i.product_id = p.product_id  
WHERE o.status = 'completed'  
GROUP BY p.category  
ORDER BY revenue DESC;  
  
→ Accessories 166.00 · Shoes 136.50 · Outerwear 84.00
```

Launch the lab

Revenue by category on Nakheel, then a film database with a table in the middle.

Suggested time • 10 minutes

[labs/D05/11-Lab-Category-Revenue/README.md](#)

DEFINITION

A bridge table sits between two tables and holds one row per link: in Pagila, one row per film per category.

Break

 Back at 7:00 PM

The TAs run the lab and the Week 2 recap from 7:00. Ask in your group channel.

Lab and Week 2 recap

TA-led. Revenue by category, then films, then the recap.

Lab: Revenue by category, with a join audit

In this lab, you will answer D04's open question with an audit after every join, then use the same method on films.

Suggested time

40 minutes

[labs/D05/11-Lab-Category-Revenue/README.md](#)

Lab: Revenue by category, with a join audit

CHECKPOINTS

C1 Revenue by category from order_items and products, audited before and after.

C2 Add orders, keep completed ones, check the total against orders alone.

C3 Pagila: films per category. Why 2,367 rows, not 1,000?

HINT

Write the joins and check the audit before adding WHERE and GROUP BY.

STRETCH

The same join on thelook_ecommerce, after a key-uniqueness check.

Week 2 Mash recap

Six to eight questions on the week, ungraded.

Suggested time • 15 minutes

CLOSE

What you can do now

- Explain why tables are split
- Run the Five Checks
- Choose INNER or LEFT JOIN
- Audit rows and totals after a join

NEXT

Keep your three .sql files for Assignment 1. D06: CTEs.